

Practica - 9 Funciones y Clases: El camino a la Modularización {#sec-funciones_clases}

Luis Gálvez

Tabla de contenidos

1 Proposito de la practica	1
1.1 Ejercicio 1: El cazador de coordenadas (Funciones y abstracción)	1
1.1.1 Desarrollo	1
1.2 Ejercicio 2: Modelando una ruta espacial (Clases e interacción)	3
1.2.1 Desarrollo	3
1.3 Preguntas	6
1.3.1 Desarrollo	7

1 Proposito de la practica

1.1 Ejercicio 1: El cazador de coordenadas (Funciones y abstracción)

En geomática, a menudo necesitamos saber cuál es la “caja delimitadora” (*Bounding Box*) de un conjunto de puntos para encuadrar un mapa.

- Escribe una función pura llamada `calcular_bbox` que reciba una lista de múltiples coordenadas (latitud, longitud).
- La función debe recorrer la lista internamente y devolver cuatro valores: la latitud mínima, la latitud máxima, la longitud mínima y la longitud máxima.
- Asegúrate de que la función incluya una cadena de documentación (Docstring) o comentarios claros explicando qué entra y qué sale.

1.1.1 Desarrollo

```
# Definir la Funcion
def calcular_bbox(coordenadas):

    #Calcula la Bounding Box (BBBox) de un conjunto de coordenadas.
```

```

# Extraer listas separadas de latitudes y longitudes
latitudes = []
longitudes = []

# Recorrer las coordenadas
for latitud, longitud in coordenadas:
    latitudes.append(latitud)
    longitudes.append(longitud)

# Calcular extremos espaciales
lat_min = min(latitudes)
lat_max = max(latitudes)
lon_min = min(longitudes)
lon_max = max(longitudes)

# Retornar resultados
return lat_min, lat_max, lon_min, lon_max

#-----
coordenadas_cundinamarca = [
    (4.7110, -74.0721), # Bogotá
    (4.9310, -74.2690), # Zipaquirá
    (4.8143, -74.3540), # Facatativá
    (4.3065, -74.8020), # Girardot
    (5.0219, -73.9950)  # Ubaté
]

#-----
# Ejecutar función
bbox = calcular_bbox(coordenadas_cundinamarca)

#-----
# Mostrar resultados
print("Bounding Box de las coordenadas:")
print(f"Latitud mínima : {bbox[0]}")
print(f"Latitud máxima : {bbox[1]}")
print(f"Longitud mínima: {bbox[2]}")
print(f"Longitud máxima: {bbox[3]}")

```

```

Bounding Box de las coordenadas:
Latitud mínima : 4.3065
Latitud máxima : 5.0219
Longitud mínima: -74.802
Longitud máxima: -73.995

```

1.2 Ejercicio 2: Modelando una ruta espacial (Clases e interacción)

Vamos a llevar los objetos al siguiente nivel haciendo que interactúen entre sí. Toma el código base del `PuntoEspacial` que desarrollamos en este capítulo y crea una nueva estructura superior:

- Crea una clase (o sistema S3 / struct en Julia) llamada `RutaGeografica`.
- Su constructor debe recibir un nombre (ej. “Ruta de los Andes”) y una lista vacía que almacenará objetos del tipo `PuntoEspacial`.
- Escribe un método `agregar_punto()` que reciba un objeto `PuntoEspacial` y lo guarde en la lista interna de la ruta.
- Escribe un método `medir_distancia_total()` que use un ciclo para sumar la distancia de todos los tramos de la ruta, llamando al método `distancia_hacia()` que los puntos ya tienen integrado.

1.2.1 Desarrollo

```
# DEFINICIÓN DE LA ESTRUCTURA PuntoEspacial
# Struct para representar un punto geográfico
struct PuntoEspacial
    nombre::String
    latitud::Float64
    longitud::Float64
end

#-----
# MÉTODO distancia_hacia()

"""
Calcula la distancia euclidiana simple entre
dos objetos PuntoEspacial.
"""
function distancia_hacia(
    origen::PuntoEspacial,
    destino::PuntoEspacial
)

    dx = origen.longitud - destino.longitud
    dy = origen.latitud - destino.latitud

    return sqrt(dx^2 + dy^2)
end

#-----
# DEFINICIÓN DE RutaGeografica
```

```

mutable struct RutaGeografica
    nombre::String
    puntos::Vector{PuntoEspacial}
end

#-----
# CONSTRUCTOR PERSONALIZADO

"""
Crea una nueva RutaGeografica con
una lista vacía de puntos espaciales.
"""
function RutaGeografica(nombre::String)

    return RutaGeografica(
        nombre,
        PuntoEspacial[]
    )
end

#-----
# MÉTODO agregar_punto()

"""
Agrega un objeto PuntoEspacial
a la ruta geográfica.
"""
function agregar_punto!(
    ruta::RutaGeografica,
    punto::PuntoEspacial
)

    push!(ruta.puntos, punto)
end

#-----
# MÉTODO medir_distancia_total()

"""
Recorre todos los tramos de la ruta
y suma la distancia entre puntos consecutivos.
"""

```

```

function medir_distancia_total(
    ruta::RutaGeografica
)

    distancia_total = 0.0

    # Recorrer pares consecutivos
    for i in 1:(length(ruta.puntos) - 1)

        punto_actual = ruta.puntos[i]
        siguiente_punto = ruta.puntos[i + 1]

        distancia_total += distancia_hacia(
            punto_actual,
            siguiente_punto
        )
    end

    return distancia_total
end

```

```

#-----
# Crear puntos espaciales
p1 = PuntoEspacial(
    "Bogotá",
    4.7110,
    -74.0721
)

p2 = PuntoEspacial(
    "Zipaquirá",
    5.0221,
    -74.0048
)

p3 = PuntoEspacial(
    "Villa de Leyva",
    5.6340,
    -73.5248
)

# Crear ruta
ruta_andes = RutaGeografica(
    "Ruta de los Andes"
)

```

```

)

# Agregar puntos a la ruta
agregar_punto!(ruta_andes, p1)
agregar_punto!(ruta_andes, p2)
agregar_punto!(ruta_andes, p3)

# Calcular distancia total
distancia_total = medir_distancia_total(
    ruta_andes
)

#-----
# Mostrar resultados
println("Ruta: ", ruta_andes.nombre)

println("\nPuntos registrados:")
for punto in ruta_andes.puntos
    println(
        punto.nombre,
        " (",
        punto.latitud,
        ", ",
        punto.longitud,
        ")"
    )
end

println(
    "\nDistancia total aproximada: ",
    round(distancia_total, digits=4),
    " grados"
)

```

1.3 Preguntas

1. **Sobre el Ejercicio 1 (Regla DRY y Pureza):** Explica con tus propias palabras qué es la regla DRY (*Don't Repeat Yourself*). ¿Por qué es un riesgo para la mantenibilidad de un proyecto copiar y pegar la gran fórmula matemática de Haversine en múltiples partes de tu código en lugar de usar la función que creaste?
2. **Sobre el Ejercicio 2 (Paradigmas de POO):** Si resolviste (o resolvieras) este ejercicio en **Python**, ¿dónde “viven” físicamente los métodos analíticos como `medir_distancia_total()`? Contrasta esta filosofía con el enfoque del **Sistema S3 de R**. Usa la analogía del

“Molde” vs el “Post-it” vista en clase para explicarlo.

3. **Pregunta General (Eficiencia y Motores):** Imagina que tu **RutaGeografica** tiene 5 millones de puntos recolectados por un GPS y necesitas calcular la distancia total. Si usarás **Julia**, explica brevemente cómo la combinación del **Despacho Múltiple** y la **Compilación JIT** lograría ejecutar este cálculo masivo mucho más rápido que el intérprete estándar de R.

1.3.1 Desarrollo

1. La regla establece que en el flujo de trabajo se debe propender por evitar repetir procesos que ya se hayan ejecutado, de ahí la utilidad de declarar una vez una función y llamarla cuando es necesario, en vez de reescribirla para cada uso, cosa que ocupa más espacio y reduce la eficiencia del proceso.

La fórmula de Haversine se utiliza para calcular la distancia más corta (círculo máximo) entre dos puntos sobre una esfera (como la Tierra) a partir de su latitud y longitud. Si se copia y pega la fórmula de Haversine en múltiples partes del proyecto, se crean varios riesgos de mantenibilidad. Por ejemplo, si se descubre un error o se necesita mejorar la precisión del cálculo, se tendría que corregir manualmente cada copia, aumentando la probabilidad de inconsistencias. Segundo, diferentes versiones podrían terminar ligeramente modificadas sin darse cuenta, produciendo resultados espaciales distintos para la misma distancia.

2. En Object-oriented programming de Python, los métodos “viven” físicamente dentro del molde de la clase. Es decir, la clase define tanto los datos como los comportamientos asociados. Por ejemplo, `medir_distancia_total()` existe dentro del bloque `class RutaGeografica`, y cada objeto creado a partir de ese molde hereda automáticamente ese comportamiento. A manera de analogía se podría decir que Python funciona como un molde industrial completo, donde el molde ya trae incorporadas las instrucciones sobre qué puede hacer el objeto. Cada instancia nace sabiendo calcular distancias, agregar puntos o imprimir resultados porque esos métodos están pegados permanentemente al diseño de la clase.

En R el objeto realmente es solo una lista común con un “Post-it” que dice “PuntoEspacial” o “RutaGeografica”. Los métodos no viven dentro del objeto, sino afuera, como funciones independientes que R busca dinámicamente usando `UseMethod()`.

3. En Julia, la combinación de Despacho Múltiple y Compilación JIT (Just-In-Time) permite que cálculos masivos se ejecuten muy cerca de la velocidad de lenguajes compilados como C. El Despacho Múltiple selecciona automáticamente la versión más eficiente de una función según

los tipos exactos de los objetos involucrados, mientras que la compilación JIT traduce el código a instrucciones máquina optimizadas en tiempo de ejecución. Esto evita gran parte de la sobrecarga interpretada que sí tiene R, especialmente dentro de ciclos grandes y operaciones numéricas intensivas.